

Sarah Benavides

CS 362 Lab 5

2/24/2026

Data Acquisition and Analysis

Part 1: Non-technical summary

In this lab, we practiced using fundamental techniques of data acquisition, sanitization, and data recovery. We began by revisiting the creation of hash values using a Windows command prompt, PowerShell, and a Linux interface. This step was crucial to begin with because we wanted to make sure the data we were analyzing was exactly the same as when we originally collected it. The second and third parts of this lab set us up for success in our data recovery journey by taking us step-by-step through creating and mounting a virtual hard disk within the virtual machine, as well as effectively sanitizing the media volume before wiping the contents of the partition.

The last few parts of this lab focused on image carving and data recovery using four different tools. When you delete a file, it technically isn't gone forever. Your system typically hides the name of the file and moves it to a temporary holding area while the data stays on the hard drive until it is overwritten with something else. We tested the following four tools attempting to recover data: recoverjpeg, foremost, scalpel, and bulk_extractor. Each of these tools performs slightly different tasks. For example, the recoverjpeg tool only recovers jpeg files, while the foremost and scalpel tools are capable of recovering many different types of file formats, including jpg, gif, png, avi, mov, etc. The last tool we tested was bulk_extractor, and this allows us to recover an even larger list of information, such as email addresses, encryption keys, domain names, credit card numbers, and many more. Overall, this lab led us effectively through the process of handling digital evidence from start to finish.

Part 2: Technical summary

To begin this lab, we revisited obtaining hash functions using Windows and Linux virtual machines. To access the .jpg file saved on my computer through the terminal, I had to change the directory to **C:\Users\sarah\OneDrive\Documents\Digital Forensics** within the Windows command prompt. In order to compute the hash value of a file, we input the command **certutil -hashfile cat1.jpg sha384**.

```
C:\Windows\System32>cd C:\Users\sarah\OneDrive\Documents\Digital Forensics
C:\Users\sarah\OneDrive\Documents\Digital Forensics>certutil -hashfile cat1.jpg sha384
SHA384 hash of cat1.jpg:
356b49fc64871572f52516f8b0d6554191da3d338c4e64d6cf1ec7e72429fdc1975b8bc7e7c1f7a78a7d5f7f73233776
CertUtil: -hashfile command completed successfully.
```

By using the following command, we are able to compute hash values for a list of .jpg files within my computer: **FORFILES /M *.jpg /C "cmd /C certutil -hashfile @FILE"**.

```
C:\Users\sarah\OneDrive\Documents\Digital Forensics>FORFILES /M *.jpg /C "cmd /C certutil -hashfile @FILE"
SHA1 hash of cat1.jpg:
3866b945d19ef4ba7768f60c40f304c05ad29411
CertUtil: -hashfile command completed successfully.
SHA1 hash of cat2.jpg:
e35b43d9a3d89d71d54e470fb1152c3d71b4b39b
CertUtil: -hashfile command completed successfully.
SHA1 hash of dog.jpg:
eaa528b45ceff0463af4050fb690c4607aa02f5a
CertUtil: -hashfile command completed successfully.
```

The commands used to obtain hash values of files within Windows PowerShell and Linux are very different than the Windows command prompt. After opening Windows PowerShell, I used **SET-LOCATION -PATH "C:\Users\sarah\OneDrive\Documents\Digital Forensics"** to change the directory. I input the following command **Get-FileHash -A SHA1 *.jpg** to obtain the hash value for a group of .jpg files. The output lists which algorithm was used, the hash values, and the path of the files.

```
PS C:\WINDOWS\system32> SET-LOCATION -PATH "C:\Users\sarah\OneDrive\Documents\Digital Forensics"
PS C:\Users\sarah\OneDrive\Documents\Digital Forensics> Get-FileHash -A SHA1 *.jpg
```

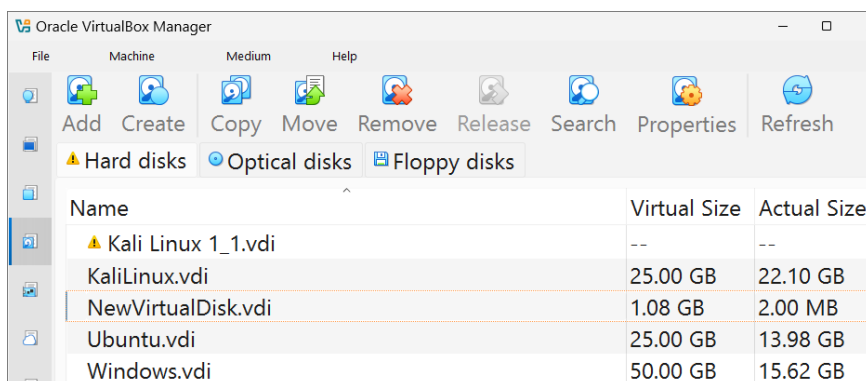
Algorithm	Hash	Path
SHA1	3866B945D19EF4BA7768F60C40F304C05AD29411	C:\User...
SHA1	E35B43D9A3D89D71D54E470FB1152C3D71B4B39B	C:\User...
SHA1	EAA528B45CEFF0463AF4050FB690C4607AA02F5A	C:\User...

For Linux, I used the following command to hash the word "hello". I used the command **printf hello | sha1sum** for this task. The second command I input into the Linux terminal was **openssl dgst -sha3-224 *.txt**. This command creates hash values for .txt files within the directory being used.

```
(kali@kali)-[~]
$ printf hello | sha1sum
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -

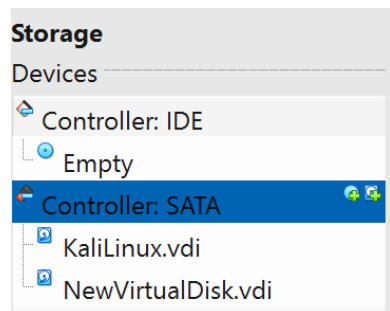
(kali@kali)-[~]
$ openssl dgst -sha3-224 *.txt
SHA3-224(file.txt)= c5a7ca6cf94f3a36fdaff92e3ddba7a52980f8b87023c169dbb6a786
```

In the second part of this lab, we focused on creating a virtual hard drive and attaching it to the virtual machine using Oracle VirtualBox. To begin, I opened the virtual media manager in VirtualBox and clicked the "CREATE" button to begin creating a new virtual media. After following all the steps of selecting the file name, type, and size, I clicked finish to create the new virtual disk named "NewVirtualDisk".



Name	Virtual Size	Actual Size
⚠️ Kali Linux 1_1.vdi	--	--
KaliLinux.vdi	25.00 GB	22.10 GB
NewVirtualDisk.vdi	1.08 GB	2.00 MB
Ubuntu.vdi	25.00 GB	13.98 GB
Windows.vdi	50.00 GB	15.62 GB

The next step in this part of the lab walked us through how to attach the new virtual hard disk to our existing Linux virtual machine.

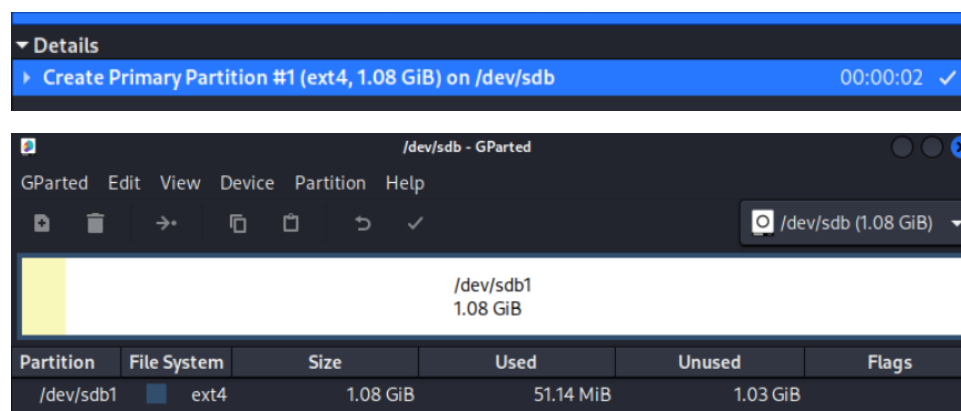


After attaching the hard disk we created in the previous step to the Linux virtual machine, I rebooted it and entered the following commands: **sudo lshw -class disk -short** and **sudo lshw -class volume -short**. The tool **lshw** was not installed on this system because the output I received was “command not found”. Once I installed **lshw**, both commands worked. Both commands display a list of all storage devices connected to the system, including their size, device name, class, and a brief description. The difference between the two commands is that the first lists the class disks and the second lists the class volume.

```
(kali@kali)-[~]
$ sudo lshw -class disk -short
H/W path          Device          Class          Description
-----
/0/100/1.1/0.0.0  /dev/cdrom      disk           CD-ROM
/0/100/d/0        /dev/sda        disk           26GB VBOX HARDDISK
/0/100/d/1        /dev/sdb        disk           1162MB VBOX HARDDISK

(kali@kali)-[~]
$ sudo lshw -class volume -short
H/W path          Device          Class          Description
-----
/0/100/d/0/1      /dev/sda1       volume         23GiB EXT4 volume
```

The next task we concentrated on was partitioning the disk **/dev/sdb** using the **gparted** visual program. To start, I typed **sudo gparted** in the terminal, and another window appeared, allowing me to create a partition table for the **/dev/sdb** drive. I followed the subsequent set of instructions to create a primary partition and applied all operations to the system.



After closing the **gparted** utility, I input the following command we used previously into the terminal: **sudo lshw -class volume -short**. When comparing this output to the output we received before, the only difference shown is that the new partition **/dev/sdb1** is listed.

```
(kali@kali)-[~]
$ sudo lshw -class volume -short
[sudo] password for kali:
H/W path      Device      Class      Description
-----
/0/100/d/0/1  /dev/sda1   volume     23GiB EXT4 volume
/0/100/d/1/1  /dev/sdb1   volume     1107MiB EXT4 volume
```

By using the following command, we are able to display /dev/sdb1 volume's name: **sudo tune2fs -l /dev/sdb1**.

```
L$ sudo tune2fs -l /dev/sdb1
tune2fs 1.47.2 (1-Jan-2025)
Filesystem volume name: <none>
Last mounted on: <not available>
Filesystem UUID: e675d6e2-5f3b-467a-9e5e-a4921ed92f40
Filesystem magic number: 0xEF53
Filesystem revision #: 1 (dynamic)
Filesystem features: has_journal ext_attr resize_inode dir_index orphan_
file filetype extent flex_bg metadata_csum_seed sparse_super large_file huge_
file dir_nlink extra_isize metadata_csum
Filesystem flags: signed_directory_hash
Default mount options: user_xattr acl
Filesystem state: clean
Errors behavior: Continue
Filesystem OS type: Linux
Inode count: 70848
Block count: 283392
Reserved block count: 14169
Overhead clusters: 12993
Free blocks: 270324
Free inodes: 70836
First block: 0
Block size: 4096
Fragment size: 4096
Reserved GDT blocks: 69
Blocks per group: 32768
Fragments per group: 32768
Inodes per group: 7872
Inode blocks per group: 492
Flex block group size: 16
Filesystem created: Thu Feb 19 16:58:26 2026
Last mount time: n/a
Last write time: Thu Feb 19 16:58:26 2026
Mount count: 0
Maximum mount count: -1
Last checked: Thu Feb 19 16:58:26 2026
Check interval: 0 (<none>)
Lifetime writes: 593 kB
Reserved blocks uid: 0 (user root)
Reserved blocks gid: 0 (group root)
First inode: 11
Inode size: 256
Required extra isize: 32
```

```
Desired extra isize: 32
Journal inode: 8
Default directory hash: half_md4
Directory Hash Seed: fad70952-ec06-495a-965d-09fb371ef259
Journal backup: inode blocks
Checksum type: crc32c
Checksum: 0x0c2d0b17
Checksum seed: 0x1239431b
Orphan file inode: 12
```

Since the volume does not have a name, the UUID is displayed as its name. The volume name is **e675d6e2-5f3b-467a-9e5e-a4921ed92f40**. When using the **df** command to display the mounting locations of the volumes, the partition /dev/sdb1 did not appear.

```
L$ df
Filesystem      1K-blocks    Used Available Use% Mounted on
udev            1900784        0   1900784   0% /dev
tmpfs           401344       1008   400336    1% /run
/dev/sda1       24253528 15511244   7484916   68% /
tmpfs           2006700         4   2006696    1% /dev/shm
tmpfs           2006704       176   2006528    1% /tmp
none            1024         0       1024   0% /run/credentials/getty@tty1.
service
tmpfs           401340        104   401236    1% /run/user/1000
none            1024         0       1024   0% /run/credentials/systemd-jou
rnald.service
```

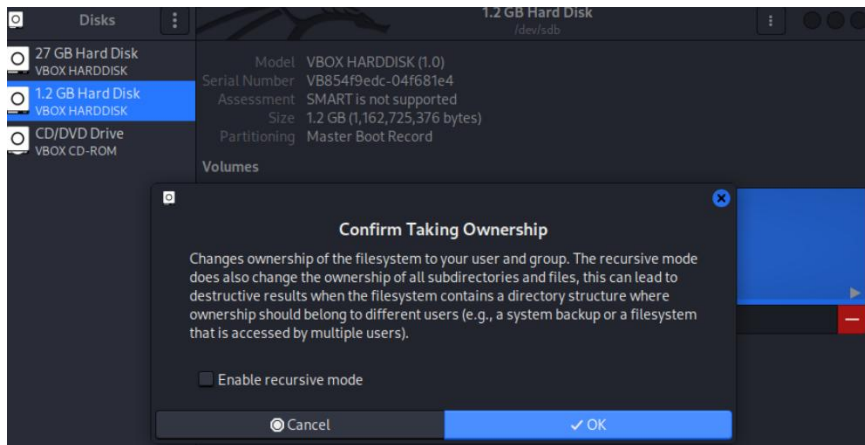
This occurred because I have not mounted the volume to the desktop yet. This next step will help with completing that task. I used the following command to change the volume's name: **sudo tune2fs -L a1forensics1g**

/dev/sdb1. After successfully making the name change, I rebooted the Linux virtual machine. Once the virtual machine came back up, I clicked on the **a1forensics1g** drive on the desktop and mounted it. Following all those changes, I tested the **df** command again to see if the volume was mounted, and it was successful.

```
(kali@kali)-[~]
$ df
Filesystem      1K-blocks    Used Available Use% Mounted on
udev            1900784         0   1900784  0% /dev
tmpfs           401344      1012    400332  1% /run
/dev/sda1       24253528 15511252   7484908  68% /
tmpfs           2006700         4   2006696  1% /dev/shm
none            1024         0     1024  0% /run/credentials/systemd-jou
rnald.service
tmpfs           2006704         8   2006696  1% /tmp
none            1024         0     1024  0% /run/credentials/getty@tty1.
service
tmpfs           401340      104    401236  1% /run/user/1000
/dev/sdb1       1081596      300   1008236  1% /run/media/kali/a1forensics1
g
```

When dealing with partitions,

we can use the **gnome-disk-utility** to take ownership of a specific partition. This task helps the user avoid writing errors.



In the third part of this lab, we are attempting to sanitize target media. To start, I input the command **sudo dd if=/dev/random of=/dev/sdb1 bs=1M status=progress** in order to sanitize the media volume before copying any content to it and showing the progress of the wiping operation simultaneously.

```
(kali@kali)-[~]
$ sudo dd if=/dev/random of=/dev/sdb1 bs=1M status=progress
[sudo] password for kali:
1136656384 bytes (1.1 GB, 1.1 GiB) copied, 38 s, 29.8 MB/s
dd: error writing '/dev/sdb1': No space left on device
1108+0 records in
1107+0 records out
1160773632 bytes (1.2 GB, 1.2 GiB) copied, 38.8631 s, 29.9 MB/s
```

Once that stage is complete, I

used the command **sudo dcfldd pattern=AAAA of=/dev/sdb1 bs=1M** to wipe the content of the partition with the pattern “AAAA”. Since the partition was only 1Gb, there was no more room on the device to complete the task once all the bytes were overwritten.

```
(kali@kali)-[~]
$ sudo dcfldd pattern=AAAA of=/dev/sdb1 bs=1M
1024 blocks (1024Mb) written.dcfldd:: No space left on device
```

To complete this part of the lab, I used **gparted** and all the previous steps to create a new partition.

In the fourth part of this lab, we experimented with Linux image carving using the **recoverjpeg** tool. There were two folders listed, and I downloaded both. Right after downloading, I unzipped and copied the contents of the folders to the partition /dev/sdc1 using the commands **sudo mv 101/ /run/media/kali/c1forensics1g/** and **sudo mv MICC-F220/ /run/media/kali/c1forensics1g/**.

```
(kali@kali)-[~/Downloads]
$ sudo mv 101/ /run/media/kali/c1forensics1g/
[sudo] password for kali:

(kali@kali)-[~/Downloads]
$ sudo mv MICC-F220/ /run/media/kali/c1forensics1g/
```

After moving the folders to the /dev/sdc1 partition, I changed the working folder (Downloads) to the mounting folder (/run/media/kali/c1forensics1g/). I completed this step using the command **cd /run/media/kali/c1forensics1g/**. To check what exactly is in the new working folder, I used the command **ls** to display the files.

```
(kali@kali)-[~/Downloads]
$ cd /run/media/kali/c1forensics1g/

(kali@kali)-[/run/media/kali/c1forensics1g]
$ ls
101  lost+found  MICC-F220
```

Our main goal in these next few steps is to delete all the .jpg files located in the folders we downloaded in the previous step. We begin by changing the working folder using the command **cd MICC-F220**. Now that we are within this folder, I am checking how many .jpg files there are within it by using the command **ls -l | grep .jpg | wc -l**.

```
(kali@kali)-[/run/media/kali/c1forensics1g]
$ cd MICC-F220

(kali@kali)-[/run/media/kali/c1forensics1g/MICC-F220]
$ ls -l | grep .jpg | wc -l
220
```

In the folder MICC-F220, there are 220 .jpg files. Inputting the command **sudo rm *.jpg** will remove all .jpg files within this folder.

```
(kali@kali)-[/run/media/kali/c1forensics1g/MICC-F220]
$ sudo rm *.jpg
[sudo] password for kali:

(kali@kali)-[/run/media/kali/c1forensics1g/MICC-F220]
$ ls -l | grep .jpg | wc -l
0
```

After I removed all .jpg files, I entered the command to check how many .jpg files there were within this folder, and the result was zero. I repeated these steps for the other folder **101/**, and received similar outputs besides the number of .jpg files that were in the folder.

```

(kali㉿kali)-[/run/media/kali/c1forensics1g/MICC-F220]
$ cd ../101

(kali㉿kali)-[/run/media/kali/c1forensics1g/101]
$ ls -l | grep .jpg | wc -l
7

(kali㉿kali)-[/run/media/kali/c1forensics1g/101]
$ sudo rm *.jpg

(kali㉿kali)-[/run/media/kali/c1forensics1g/101]
$ ls -l | grep .jpg | wc -l
0

```

I also used to the command `ls -l | wc -l` to find how many non-jpeg files were in the 101/ folder.

```

(kali㉿kali)-[/run/media/kali/c1forensics1g/101]
$ ls -l | wc -l
974

```

Once we deleted the .jpg files from both folders, we want to try to recover them into a new folder. To create that new folder, I input the command `mkdir jpegrecovery &&` and input the command `cd ../jpegrecovery`. After changing the working folder, I checked how many files were in the folder and the total was zero.

```

(kali㉿kali)-[/run/media/kali/c1forensics1g]
$ mkdir jpegrecovery 86
cmdand> cd ../jpegrecovery
cd: no such file or directory: ../jpegrecovery

(kali㉿kali)-[/run/media/kali/c1forensics1g]
$ cd jpegrecovery

(kali㉿kali)-[/run/media/kali/c1forensics1g/jpegrecovery]
$ ls -l
total 0

```

In order to perform the recovery of the .jpg files, I installed **recoverjpeg**. After everything was successfully installed, I used **recoverjpeg** to begin recovering the deleted files. I used the command `sudo recoverjpeg /dev/sdc1 -o ./jpegrecovery` to recover the files and place them into the new **jpegrecovery** folder. As soon as the recovery finished, I input the command `ls -l ./jpegrecovery/*.jpg | wc -l` to count and display the number of files that were recovered.

```

(kali㉿kali)-[/run/media/kali/c1forensics1g]
$ sudo recoverjpeg /dev/sdc1 -o ./jpegrecovery
Restored 228 pictures

(kali㉿kali)-[/run/media/kali/c1forensics1g]
$ ls -l ./jpegrecovery/*.jpg | wc -l
228

```

In the fifth part of this lab, we examine how different data recovery is using the foremost tool instead of **recoverjpeg**. **Recoverjpeg** is used solely to recover .jpg files, but the **foremost** tool can recover a long list of files, including jpg, gif, png, avi, mov, exe, pdf, doc, and zip. I started this part off by downloading the **foremost** tool. After successfully installing the tool, I want to delete all the files and folders in the /dev/sdb1 partition. The first command I entered was `cd ~ && mkdir evidencesdb1` to change the current directory to the home directory while creating a folder called **evidencesdb1**. In the next command, I input `sudo dcfldd if=/dev/sdb1`

of=./evidencesdb1/sdb1image.dd hash=sha1 hashlog=./evidencesdb1/hashlog.log in order to create an image and store it on the main partition.

```
(kali@kali)-[/run/media/kali/c1forensics1g]
$ cd ~ && mkdir evidencesdb1

(kali@kali)-[~]
$ sudo dcfldd if=/dev/sdb1 of=./evidencesdb1/sdb1image.dd hash=sha1 hashlog
=./evidencesdb1/hashlog.log
35328 blocks (1104Mb) written.
35424+0 records in
35424+0 records out
```

Once I finished creating and storing the new image in the evidencesdb1 folder, I used the command **ls evidencesdb1** to display what was inside that folder. The next command I used was **cat ./evidencesdb1/hashlog.log** to list the hash value that was created for that file.

```
(kali@kali)-[~]
$ ls evidencesdb1
hashlog.log  sdb1image.dd

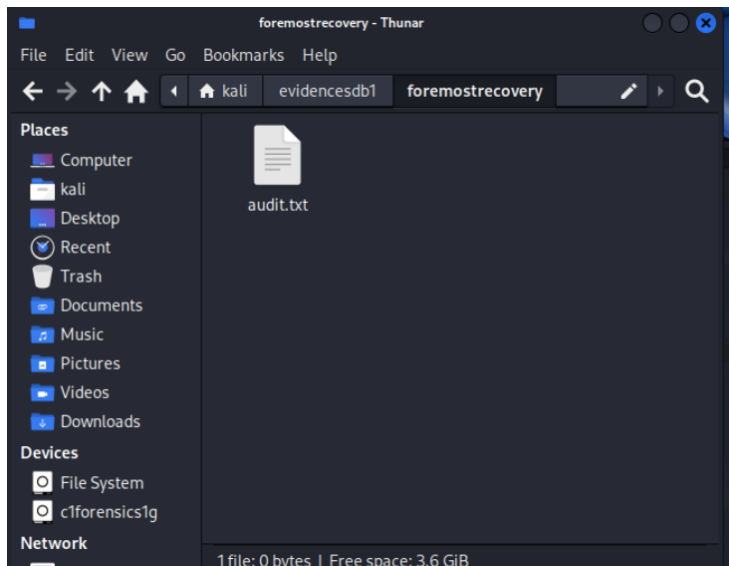
(kali@kali)-[~]
$ cat ./evidencesdb1/hashlog.log

Total (sha1): 3fa77f3481b612544e727288e816eb965addddb1
```

The last step in this part to complete is recovering the content of the partition using the foremost tool. To do this I used the command **sudo foremost -t all -i ./evidencesdb1/sdb1image.dd -o ./evidencesdb1/foremostrecovery**.

```
(kali@kali)-[~]
$ sudo foremost -t all -i ./evidencesdb1/sdb1image.dd -o ./evidencesdb1/foremostrecovery
Processing: ./evidencesdb1/sdb1image.dd
|*****|
```

When I navigated to the output folder, unfortunately the only file in there was audit.txt.



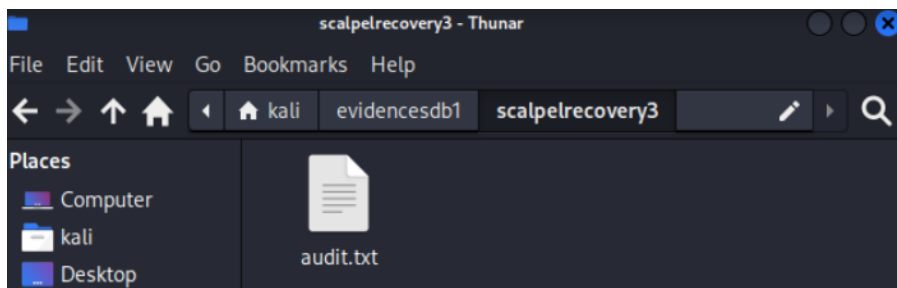
In the second-to-last part of this lab, we focused on recovering data using the scalpel tool. Scalpel is similar to the foremost tool when it comes to the tasks it can perform. After I downloaded the scalpel tool, I visited the configuration file **/etc/scalpel/scalpel.conf**. Within that file, I searched for specific file types and uncommented them. Once that step was complete,

I input the command **sudo scalpel -o /home/kali/evidence/sdb1/scalpelrecovery3 /home/kali/evidencesdb1/sdb1image.dd**. I had to add the “/home/kali” because the terminal wasn’t recognizing my input when I entered the example command within the lab document.

```
$ sudo scalpel -o /home/kali/evidencesdb1/scalpelrecovery3 /home/kali/evidencesdb1/sdb1image.dd
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/kali/evidencesdb1/sdb1image.dd"

Image file pass 1/2.
/home/kali/evidencesdb1/sdb1image.dd: 100.0% |*****| 1.1 GB 00:00 ETA
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built. Workload:
GIF with header "\x66\x69\x6c\x65\x73" and footer "\x28\x76\x65\x72\x79" → 0 files
gif with header "\x47\x49\x46\x38\x37\x61" and footer "\x00\x3b" → 0 files
gif with header "\x47\x49\x46\x38\x39\x61" and footer "\x00\x3b" → 0 files
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" → 0 files
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" → 0 files
Carving files from image.
Image file pass 2/2.
/home/kali/evidencesdb1/sdb1image.dd: 100.0% |*****| 1.1 GB 00:00 ETA
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 0, elapsed = 11 seconds.
```



After inputting the correct command, the tool worked. Even though I copied a few images to the drive and deleted them before creating the sdb1 image, the only file in the output directory was audit.txt, which was the same as using the foremost tool. I also verified I was using the current drive on my computer, but I did not have any luck with the output changing.

The last part of this lab explores recovering data and information retrieval using bulk_extractor. This tool is handy when things like email addresses, encryption keys, domain names, credit card numbers, etc. need to be recovered. I input the following command to extract more information from the suspect sdb1image.dd image: **sudo bulk_extractor -o ./evidencesdb1/bulk ./evidencesdb1/sdb1image.dd**.

```

(kali@kali)-[~]
└─$ sudo bulk_extractor -o ./evidencesdb1/bulk ./evidencesdb1/sdb1image.dd
[sudo] password for kali:
mkdir ./evidencesdb1/bulk
bulk_extractor version: 2.1.1
Input file: "./evidencesdb1/sdb1image.dd"
Output directory: "./evidencesdb1/bulk"
Disk Size: 1160773632
Scanners: aes base64 elf evtx exif facebook find gzip httplogs json kml_carve
d mxsml net ntfsindx ntfslogfile ntfsmft ntfsusn pdf rar sqlite utmp vcard_ca
rved windirs winlnk winpe winprefetch zip accts email gps
Threads: 2
going multi-threaded... ( 2 )
bulk_extractor Thu Feb 19 23:31:09 2026

available_memory: 3133796352
bytes_queued: 0
depth0_bytes_queued: 0
depth0_sbufs_queued: 0
elapsed_time: 0:00:00
estimated_date_completion: 2026-02-19 23:31:08
estimated_time_remaining: n/a
fraction_read: 0.000000 %
max_offset: 0
sbufs_created: 0
sbufs_queued: 0
sbufs_remaining: 0
tasks_queued: 0
thread_count: 2
>.....|

bulk_extractor Thu Feb 19 23:31:10 2026

available_memory: 3107876864
bytes_queued: 83886080
depth0_bytes_queued: 83886080
depth0_sbufs_queued: 4
elapsed_time: 0:00:01
estimated_date_completion: 2026-02-19 23:32:34
estimated_time_remaining: 0:01:24
fraction_read: 1.445348 %
max_offset: 0

```

```

max_offset: 0
sbufs_created: 98306
sbufs_queued: 4
sbufs_remaining: 2
tasks_queued: 2
thread-1: 0: net (20971520 bytes)
thread-2: 0: email (20971520 bytes)
thread_count: 2
⇒.....|

bulk_extractor Thu Feb 19 23:31:11 2026

available_memory: 3111227392
bytes_queued: 104857600
depth0_bytes_queued: 104857600
depth0_sbufs_queued: 5
elapsed_time: 0:00:02
estimated_date_completion: 2026-02-19 23:32:26
estimated_time_remaining: 0:01:15
fraction_read: 2.890696 %
max_offset: 16777216
sbufs_created: 196611
sbufs_queued: 5
sbufs_remaining: 2
tasks_queued: 3
thread-1: 16777216: net (20971520 bytes)
thread-2: 16777216: accts (20971520 bytes)
thread_count: 2
⇒.....|

bulk_extractor Thu Feb 19 23:31:12 2026

available_memory: 3122294784
bytes_queued: 0
depth0_bytes_queued: 0
depth0_sbufs_queued: 0
elapsed_time: 0:00:03
estimated_date_completion: 2026-02-19 23:31:46
estimated_time_remaining: 0:00:34
fraction_read: 8.672087 %
max_offset: 16777216

```

```
max_offset: 16777216
sbufs_created: 196616
sbufs_queued: 0
sbufs_remaining: 1
tasks_queued: 0
thread_count: 2
=====→.....|

bulk_extractor      Thu Feb 19 23:31:13 2026

available_memory: 3098238976
bytes_queued: 20971520
depth0_bytes_queued: 20971520
depth0_sbufs_queued: 1
elapsed_time: 0:00:04
estimated_date_completion: 2026-02-19 23:31:42
estimated_time_remaining: 0:00:29
fraction_read: 13.008130 %
max_offset: 134217728
sbufs_created: 294923
sbufs_queued: 1
sbufs_remaining: 2
tasks_queued: 0
thread-1: 134217728: net (20971520 bytes)
thread_count: 2
=====→.....|

bulk_extractor      Thu Feb 19 23:31:14 2026

available_memory: 3097751552
bytes_queued: 272629760
depth0_bytes_queued: 272629760
depth0_sbufs_queued: 13
elapsed_time: 0:00:05
estimated_date_completion: 2026-02-19 23:31:23
estimated_time_remaining: 0:00:09
fraction_read: 36.133695 %
max_offset: 402653184
sbufs_created: 294938
sbufs_queued: 13
sbufs_remaining: 2

sbufs_remaining: 2
tasks_queued: 11
thread-1: 402653184: net (20971520 bytes)
thread-2: 402653184: sqlite (20971520 bytes)
thread_count: 2
=====→.....|

bulk_extractor      Thu Feb 19 23:31:15 2026

available_memory: 3097772032
bytes_queued: 629145600
depth0_bytes_queued: 629145600
depth0_sbufs_queued: 30
elapsed_time: 0:00:06
estimated_date_completion: 2026-02-19 23:31:22
estimated_time_remaining: 0:00:07
fraction_read: 46.251129 %
max_offset: 402653184
sbufs_created: 393250
sbufs_queued: 30
sbufs_remaining: 2
tasks_queued: 28
thread_count: 2
=====→.....|

bulk_extractor      Thu Feb 19 23:31:17 2026

available_memory: 3097272320
bytes_queued: 83886080
depth0_bytes_queued: 83886080
depth0_sbufs_queued: 4
elapsed_time: 0:00:07
estimated_date_completion: 2026-02-19 23:31:25
estimated_time_remaining: 0:00:08
fraction_read: 47.696477 %
max_offset: 536870912
sbufs_created: 491554
sbufs_queued: 4
sbufs_remaining: 2
tasks_queued: 2
thread-1: 536870912: net (20971520 bytes)
```

```
thread-1: 536870912: net (20971520 bytes)
thread-2: 536870912: email (20971520 bytes)
thread_count: 2
```

```
bulk_extractor Thu Feb 19 23:31:18 2026
```

```
available_memory: 3097276416
bytes_queued: 419430400
depth0_bytes_queued: 419430400
depth0_sbufs_queued: 20
elapsed_time: 0:00:08
estimated_date_completion: 2026-02-19 23:31:23
estimated_time_remaining: 0:00:05
fraction_read: 59.259259 %
max_offset: 671088640
sbufs_created: 491562
sbufs_queued: 20
sbufs_remaining: 2
tasks_queued: 18
thread-1: 671088640: aes (20971520 bytes)
thread-2: 671088640: net (20971520 bytes)
thread_count: 2
```

```
bulk_extractor Thu Feb 19 23:31:19 2026
```

```
available_memory: 3099410432
bytes_queued: 20971520
depth0_bytes_queued: 20971520
depth0_sbufs_queued: 1
elapsed_time: 0:00:09
estimated_date_completion: 2026-02-19 23:31:23
estimated_time_remaining: 0:00:04
fraction_read: 69.376694 %
max_offset: 805306368
sbufs_created: 589874
sbufs_queued: 1
sbufs_remaining: 3
tasks_queued: 0
thread-1: 805306368 release_sbuf
```

```
thread-1: 805306368 release_sbuf
thread-2: 671088640: net (20971520 bytes)
thread_count: 2
```

```
bulk_extractor Thu Feb 19 23:31:20 2026
```

```
available_memory: 3093434368
bytes_queued: 20971520
depth0_bytes_queued: 20971520
depth0_sbufs_queued: 1
elapsed_time: 0:00:10
estimated_date_completion: 2026-02-19 23:31:22
estimated_time_remaining: 0:00:02
fraction_read: 83.830172 %
max_offset: 939524096
sbufs_created: 688188
sbufs_queued: 1
sbufs_remaining: 2
tasks_queued: 0
thread-1: 939524096: net (20971520 bytes)
thread_count: 2
```

```
All data read; waiting for threads to finish...
```

```
bulk_extractor Thu Feb 19 23:31:21 2026
```

```
available_memory: 3092963328
bytes_queued: 0
depth0_bytes_queued: 0
depth0_sbufs_queued: 0
elapsed_time: 0:00:11
estimated_date_completion: 2026-02-19 23:31:21
estimated_time_remaining: 0:00:00
fraction_read: 100.000000 %
max_offset: 939524096
sbufs_created: 688198
sbufs_queued: 0
sbufs_remaining: 0
tasks_queued: 1
thread_count: 1
```

```
Phase 2. Shutting down scanners
Computing final histograms and shutting down...
Phase 3. Generating stats and printing final usage information
All Threads Finished!
Elapsed time: 12.76 sec.
Total MB processed: 1160
Overall performance: 90.98 MBytes/sec 45.49 (MBytes/sec/thread)
sbufs created: 688198
sbufs unaccounted: 0
Time producer spent waiting for scanners to process data: 0:00:07 (7.
01 seconds)
Time consumer scanners spent waiting for data from producer: 0:00:00 (0.
00 seconds)
Average time each consumer spent waiting for data from producer: 0:00:00 (0.
00 seconds)
*** More time spent waiting for workers. You need a faster CPU or more cores
for improved performance.
Total email features found: 0
```

Part 3: References

1. Lab 5: Data Acquisition and Analysis
2. [What Happens Behind the Scenes When You Delete a File?](#)